RICE UNIVERSITY

D1 · Collect: stream a text dump

Read article pages from compressed XML into a clean Parquet table, without loading the whole file.

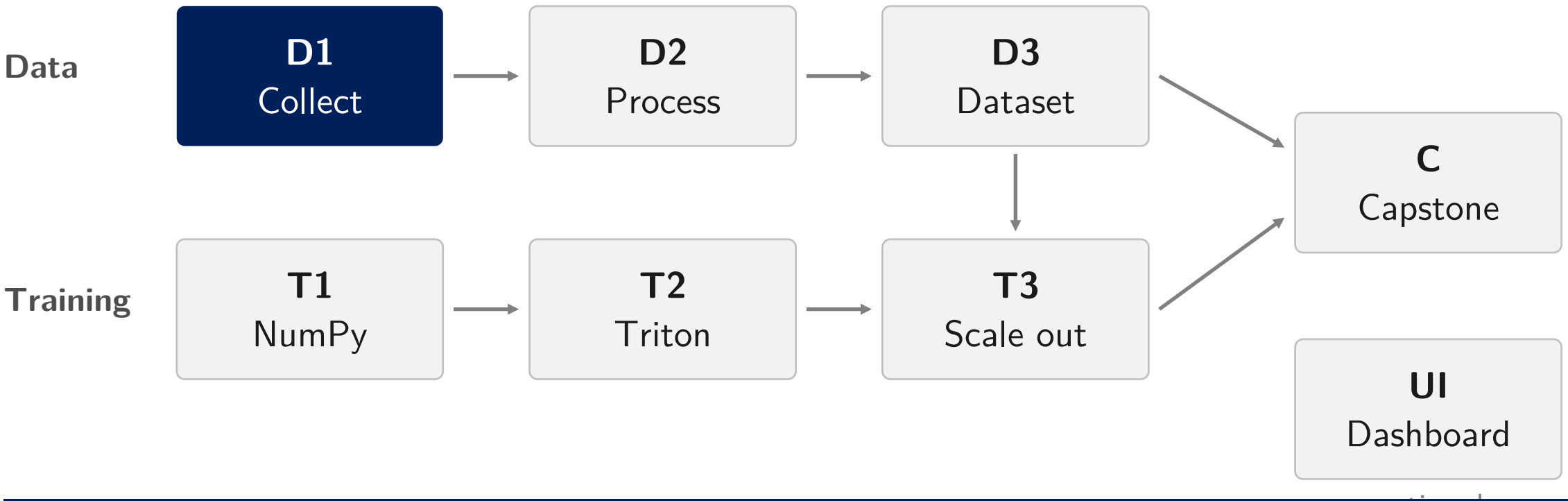
COMP 329 course staff

Rice University

Spring 2027 · Assignment D1

Release week 1 · Checkpoint week 2 · Final week 3 · 8-10 hours

Where this assignment fits



In our pipeline

This assignment: `data/tiny-wiki.xml.bz2` → **D1** → `articles.parquet`

What you will build, and why

Turn a raw XML dump into a typed table the next stage can trust, and explain every page you skip.

1 **Streaming**

Real dumps are far larger than memory. Process one page, release it, keep only a small batch.

2 **Provenance**

Every row keeps its page ID, revision, timestamp, source and license.

3 **Accountability**

Each skipped page gets one logged reason, so counts always add up.

What goes in, what comes out



Files you produce

`articles.parquet`
Accepted article records

`articles.rejected.jsonl`
One line per skipped page, with its reason

`articles.stats.json`
Counts, sizes, throughput, checksum

Streaming keeps memory bounded

1

Open stream

Pick open, gzip.open or bz2.open from the suffix; read bytes as a stream.

→

2

iterparse

Handle each </page> end event as soon as it completes.

→

3

Extract + check

Page fields, revision fields, then the first matching skip reason.

→

4

Batch

Hold at most `batch_size` accepted records.

→

5

Write + clear

Write a Parquet row group; clear the element and root references.

Key idea

Without clearing finished elements, incremental parsing can still keep the whole tree in memory.

Page identity vs. revision identity

```
<page>
  <title>Arrays</title>
  <ns>0</ns>
  <id>11</id>
  <revision>
    <id>101</id>
    <timestamp>2027-01-01…</timestamp>
    <text>Arrays store…</text>
  </revision>
</page>
```

1. `doc_id = "11"`

   Comes from the page's own <id>, not the revision's.

2. `revision_id = "101"`

   Comes from the <id> inside <revision>.

3. **Skip reasons, first match wins**

   namespace → redirect → empty → `missing_metadata`

4. `pages_seen` counts everything

   Skipped pages count too; `max_pages` limits this number.

What a correct first job produces

37	34	3
pages seen	articles kept	pages skipped

Key idea

The three skips: one redirect, one non-article namespace, one empty page. The short article and the contact-spam article stay; filtering them is D2's job.

What you write, what you get

You write

`collect(...)`

`starter/collect.py` · 7 numbered TODOs, keep the signature

Provided

- SCHEMA and `strip_markup`
- `file_hash` and `write_json`
- The lab.py launcher (spawn-safe imports)
- Public tests in `tests/test_d1.py`

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1 — 2 — 3 — 4 — 5

One page

Parse one uncompressed page; test page vs. revision IDs.

Skip + log

Counters and the rejection log; clear finished pages.

Batches

Bounded Parquet row groups; flush leftovers at the end.

Compression

Stream .xml, .gz and .bz2 in binary mode.

Safe + parallel

Temp files, rename on success, spawned workers.

```
check d1 --checkpoint
```

```
check d1
```

Check

Done when: All 5 tests pass, and the first job reports 37 / 34 / 3.

If something goes wrong

`doc_id is 10001 for page 1`

Check: You used the revision ID instead of the page ID.

Final rows disappear

Check: Flush the last, partially filled batch after parsing.

Memory grows with file length

Check: Clear completed elements and root references.

Broken XML changes the output

Check: Rename temp files only after parsing succeeds.

Two-worker run hangs

Check: Use the launcher, a spawn context, and shut the pool down in finally.

Counts do not add up

Check: Every page goes to `pages_seen`; each skip has exactly one reason.

The performance study

Do two parser workers help a 37-page file?

Predict first

Decompression, XML traversal, writing and setup stay serial. Predict before timing.

Workers 1 / 2 / 4
batch size 8

Batch size 8 / 32
one worker count

How to measure: One warmup + three repeats per setting. Report median and range of seconds, records/s, compressed MB/s and output bytes.

How you are graded

20	40	16	24
Checkpoint	Final correctness	Performance	Analysis

Correctness: schema/provenance 10 · filtering/counters 8 · streaming/compression 10 · rollback/parallel 12

Check

Done when: All 5 tests pass, and the first job reports 37 / 34 / 3.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py inputs --output runs/inputs
$ python course.py check d1 --checkpoint
$ python lab.py d1 --input data/tiny-wiki.xml.bz2 \
    --output runs/d1-first --workers 1
```

Your folder

`01_d1_collect/`

Handout:
`01_d1_collect/README.md`